Name:Gaddam Sai

Hallticket:2303A52255

Assginment:2303A52255

Task:01

Database Schema Design for an

Online Course Platform

Scenario

You are designing the backend database for an online learning platform

that manages instructors, courses, and enrolled students.

Task Description

• Use AI tools to generate SQL CREATE TABLE statements for the

following entities:

o Instructors

o Courses

o Students

• Include:

o Primary keys

o Foreign key relationships

o Appropriate data types

• Ensure the schema follows basic normalization principles.

Expected Output

• All tables are created successfully

• Foreign key constraints enforce valid relationships

• Schema is logically structured and normalized

Prompt: give me code to create table

Code:

```sql
-- 1 Instructors
CREATE TABLE instructors (
    instructor_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    expertise TEXT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- 2 Students
CREATE TABLE students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    enrolled_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- 3 Courses
CREATE TABLE courses (
    course_id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    description TEXT,
    instructor_id INTEGER,
    price REAL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (instructor_id)
        REFERENCES instructors(instructor_id)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);
```

SQL ▾

enrollment_i
1

DEBUG CONSOLE TERMINAL 8 PORTS SPELL CHECKER 8

```sql
-- 4️⃣ Enrollments (Many-to-Many)
CREATE TABLE enrollments (
    enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER,
    course_id INTEGER,
    enrolled_on DATETIME DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (student_id)
        REFERENCES students(student_id)
        ON DELETE CASCADE,

    FOREIGN KEY (course_id)
        REFERENCES courses(course_id)
        ON DELETE CASCADE,

    UNIQUE(student_id, course_id)
);
```

Output:

student_id	name	email	enrolled_at
1	Sai	sai@gmail.com	2026-03-26 09:03:42

Observation:

1. Proper Normalization Achieved

- The database is designed using **basic normalization principles (up to 3NF)**.
- Data redundancy is minimized:
    - Instructor details are stored only in the **Instructors** table.

    - Course details are stored in the **Courses** table.
    - Student data is isolated in the **Students** table.
- This ensures **no duplication of data** and better maintainability.

Task:02

Populate the Database with Sample

Records

Scenario

To test the system, the platform needs initial sample data.

Task Description

• Use AI assistance to generate INSERT statements that add:

o At least 3 instructors

o At least 4 courses

o At least 3 students

• Verify correctness using SELECT queries.

Expected Output

• Sample data inserted without errors

• SELECT * queries display the inserted records correctly

Prompt: Use AI assistance to generate INSERT statements that add:

Code:

```sql
--task 02
SELECT
    c.course_id,
    c.title,
    i.name AS instructor_name
FROM courses c
JOIN instructors i
ON c.instructor_id = i.instructor_id;
SELECT
    s.name AS student_name,
    c.title AS course_name
FROM enrollments e
JOIN students s
ON e.student_id = s.student_id
JOIN courses c
ON e.course_id = c.course_id;
SELECT
    c.title,
    COUNT(e.student_id) AS total_students
FROM courses c
LEFT JOIN enrollments e
ON c.course_id = e.course_id
GROUP BY c.course_id;
SELECT *
FROM courses
WHERE price > 800;
SELECT *
FROM students
WHERE name = 'Sai';
SELECT COUNT(*) FROM instructors;
SELECT COUNT(*) FROM students;
SELECT COUNT(*) FROM courses;
```

Output:

SQL ▾ 1 / 1 1 - 1 of 1

course_id	title	instructor_name
1	AI Course	John

SQL ▼ 1 / 1 1 - 1 of 1

student_name	course_name
Sai	AI Course

SQL ▼ 1 / 1 1 - 1 of 1

title	total_students
AI Course	1

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 1 of 1

student_id	name	email	enrolled_at
1	Sai	sai@gmail.com	2026-03-26 09:03:42

SQL ▾ 1 / 1 1 - 1 of 1

COUNT(*)
1

SQL ▾ 1 / 1 1 - 1 of 1

COUNT(*)
1

SQL ▾ 1 / 1 1 - 1 of 1

COUNT(*)
1

Observation

The database has been successfully populated with valid sample data, ensuring **data integrity, correct relationships, and accurate retrieval**, making it ready for testing and further development.

Task:03

Implement Core Database

Operations (CRUD)

Scenario

The platform administrator needs to manage course and student

information.

Task Description

Using AI-generated SQL, perform the following operations:

• Retrieve all courses offered

• Update a student's email address

• Remove a course from the database

• Verify each operation using appropriate SELECT statements

Expected Output

• Updated data is reflected correctly

• Deleted records are no longer visible

• No unintended data loss occurs

Prompt:give the code for student adminstatior table

Code:

```
--task03
SELECT * FROM courses;
UPDATE students
SET email = 'sai_new@gmail.com'
WHERE name = 'Sai';
SELECT * FROM students WHERE name = 'Sai';
DELETE FROM courses
WHERE title = 'Deep Learning';
SELECT * FROM courses;
```

Output:

SQL ▾ 1 / 1 1 - 1 of 1

course_id	title	description	instructor_id	price	created_at
1	AI Course	NULL	1	NULL	2026-03-26 09:03

SQL ▾ 1 / 1 1 - 1 of 1

student_id	name	email	enrolled_at
1	Sai	sai_new@gmail.com	2026-03-26 09:03:42

SQL ▾ 1 / 1 1 - 1 of 1

course_id	title	description	instructor_id	price	created_at
1	AI Course	NULL	1	NULL	2026-03-26 09:03

Observation:

All CRUD operations were executed successfully, with courses retrieved, student email updated, and the selected course deleted without errors.

SELECT queries confirmed that updates were reflected correctly and deleted records were no longer present.

Data integrity was maintained throughout, with no unintended data loss or constraint violations.

Task:04:

Reporting Using Aggregate Queries

Scenario

The platform management wants summary reports for decision-making.

Task Description

• Use AI tools to generate SQL queries that:

o Count the number of students enrolled in each course

o List courses with more than a specified number of

enrollments

• Analyze the query results for correctness.

Expected Output

• Accurate aggregated results

• Queries use GROUP BY and HAVING appropriately

Prompt:

Give me the code left join operations

Code:

```sql
--task04
SELECT
    c.course_id,
    c.title,
    COUNT(e.student_id) AS total_students
FROM courses c
LEFT JOIN enrollments e
ON c.course_id = e.course_id
GROUP BY c.course_id, c.title;
```

Output:

SQL ▾ 1 / 1 1 - 1 of 1

course_id	title	total_students
1	AI Course	1

Observation:

1. Aggregate queries using GROUP BY correctly counted the number of students enrolled in each course, producing accurate summaries.
2. The HAVING clause successfully filtered courses based on enrollment thresholds, showing only those exceeding the specified limit.
3. Results were consistent and reliable, confirming proper use of aggregation functions without data mismatch or duplication.

Task:05

Query Optimization with AI

Suggestions

Scenario

As the platform grows, query performance becomes critical.

Task Description

• Use AI to analyze an existing query that retrieves courses taught

by instructors from a specific country.

• Refactor the query using:

o Efficient joins

o Optional indexing suggestions

• Compare results before and after optimization.

Expected Output

• Optimized query returns the same results as the original

• Improved readability and performance for larger datasets

Prompt: • Use AI to analyze an existing query that retrieves courses taught

by instructors from a specific country.

• Refactor the query using:

Code:

```sql
--task 05
ALTER TABLE instructors ADD COLUMN country TEXT;
UPDATE instructors
SET country = 'India'
WHERE instructor_id = 1;

UPDATE instructors
SET country = 'USA'
WHERE instructor_id = 2;

UPDATE instructors
SET country = 'India'
WHERE instructor_id = 3;
SELECT
    c.course_id,
    c.title,
    i.name AS instructor_name,
    i.country
FROM courses c
INNER JOIN instructors i
ON c.instructor_id = i.instructor_id
WHERE i.country = 'India';
```

Output:

SQL ▾ 1 / 1 1 - 1 of 1

course_id	title	instructor_name	country
1	AI Course	John	India

Observation:

 The optimized query using **JOIN** improves performance compared to the subquery, especially for large datasets.

 Indexing on country and instructor_id significantly speeds up filtering and join operations.

 Both queries return the same results, but the optimized version offers better readability and scalability.